3.Write a Java program to demonstrate

a) method overloading and method overriding

Type	Input	Expected Output	Explanation
Success	add(10,5)	15	Overloaded integer method
Success	add(2.5,3.5)	6.0	Overloaded double method
Success	Child object calls overridden display()	Child display method executed	Overriding
Failure/Edge	Parent reference = Child object	Child method executed	Runtime polymorphism
Failure/Edge	Call undefined overloaded method	Compilation error	Invalid method signature

Algorithm:

1. Start.
2. Create a Calculator object.
3. Call the overloaded add(int, int) method with integer values and display the result.
4. Call the overloaded add(double, double) method with double values and display the result.
5. Create a Child object.
6. Call the display() method using the Child object to demonstrate **method overriding**.
7. Create a Parent reference and assign it a Child object.
8. Call the display() method using the Parent reference to demonstrate **runtime polymorphism** (the overridden method in Child is executed).
9. End.

Program:

```
// Parent class
class Parent {
    void display() {
        System.out.println("Parent display method executed");
    }
}

// Child class overriding display()
class Child extends Parent {
   // @Override
    void display() {
        System.out.println("Child display method executed");
    }
}

// Class demonstrating method overloading
class Calculator {

    // Overloaded method for integers
    int add(int a, int b) {
        return a + b;
    }

    // Overloaded method for doubles
    double add(double a, double b) {
```

```
        return a + b;
    }
}

public class MethodDemo {
    public static void main(String[] args) {

        Calculator calc = new Calculator();

        // Method Overloading
        System.out.println("add(10, 5) = " + calc.add(10, 5));
        System.out.println("add(2.5, 3.5) = " + calc.add(2.5, 3.5));

        // Method Overriding
        Child child = new Child();
        child.display();

        // Runtime Polymorphism
        Parent ref = new Child();
        ref.display();

        /*
         * Uncommenting the following line will cause
         * a compilation error because no matching
         * overloaded method exists.
         */
        // System.out.println(calc.add("10", "5"));
    }
        }
```

b)inheritance (single, multilevel) and interfaces (for multiple inheritance)

Type	Input	Expected Output	Explanation
Success	Single inheritance: Student extends Person	Parent and child methods accessible	Single inheritance
Success	Multilevel: Animal → Dog → Puppy	All inherited methods accessible	Multilevel inheritance
Success	Interface implementation for multiple inheritance	Interface methods executed	Multiple inheritance using interface
Failure/Edge	Interface method not implemented	Compilation error	Interface rule
Failure/Edge	Access private parent member directly	Compilation error	Encapsulation check

Algorithm:

1. **Start**.
2. Create a Student object.
3. Call the inherited showPerson() method.
4. Call the showStudent() method.
5. Display the results of **Single Inheritance**.
6. Create a Puppy object.
7. Call the inherited eat() method from Animal.

8. Call the inherited bark() method from Dog.
9. Call the weep() method from Puppy.
10. Display the results of **Multilevel Inheritance**.
11. Create a Child object.
12. Call fatherProperty() from the Father interface.
13. Call motherProperty() from the Mother interface.
14. Display the results of **Multiple Inheritance using Interfaces**.
15. Note that:
    - If a class implements an interface but does not implement all its methods, a **compilation error** occurs.
    - A child class **cannot directly access private members** of its parent class.
16. **Stop**.

Program:

```
// Parent class for Single Inheritance
class Person {
    void showPerson() {
        System.out.println("Person method executed");
    }
}

// Child class extending Person
class Student extends Person {
    void showStudent() {
        System.out.println("Student method executed");
    }
}

// Multilevel Inheritance
class Animal {
    void eat() {
        System.out.println("Animal can eat");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog can bark");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy can weep");
    }
}

// Interfaces for Multiple Inheritance
interface Father {
    void fatherProperty();
}
```

```
interface Mother {
    void motherProperty();
}

// Class implementing multiple interfaces
class Child implements Father, Mother {

    @Override
    public void fatherProperty() {
        System.out.println("Father's property method executed");
    }

    @Override
    public void motherProperty() {
        System.out.println("Mother's property method executed");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {

        // Single Inheritance
        System.out.println("Single Inheritance:");
        Student student = new Student();
        student.showPerson();   // Parent method
        student.showStudent();  // Child method

        // Multilevel Inheritance
        System.out.println("\nMultilevel Inheritance:");
        Puppy puppy = new Puppy();
        puppy.eat();   // Animal method
        puppy.bark();  // Dog method
        puppy.weep();  // Puppy method

        // Multiple Inheritance using Interfaces
        System.out.println("\nMultiple Inheritance using Interfaces:");
        Child child = new Child();
        child.fatherProperty();
        child.motherProperty();

        /*
         * Failure/Edge Cases
         */

        // 1. Interface method not implemented
        // Compilation Error Example:
        /*
        class Test implements Father {
            // fatherProperty() not implemented
        }
        */
```

```
    // 2. Access private parent member directly
    // Compilation Error Example:
    /*
    class Parent {
        private int age = 25;
    }

    class ChildClass extends Parent {
        void display() {
            System.out.println(age); // Error: age has private access
        }
    }
    */
}
    }
```